Suggested Teaching Guidelines for

Advanced Programming for AI (Java Programming)

PGCP-AI February 2026

Duration: **36** Theory hours + **30** Lab hours + **24** Self Learning hours

Objective: To introduce the student to Java programming

Prerequisites: Knowledge of programming in any language like C, C++

Evaluation method: Theory exam– 40% weightage
Lab exam – 40% weightage
Internal exam– 20% weightage

List of Books / Other training material

Courseware: No specific courseware for the module, faculty may share some coursematerials.

Reference Book:

1. Java - The Complete Reference by Herbert Schildt / Tata Mcgraw Hill Education
2. Java Server Programming (J2EE 1.7 Edition) Black Book by Dreamtech Software Team
3. Java 8 Programming Black Book by Dreamtech Press
4. Core Java: Fundamentals - Volume 1 Gary Cornell, Cay S. Horstmann/ Pearson
5. Programming in Java by Sachin Malhotra, Saurabh Choudhary / Oxford University Press
6. Core Java: Advanced Features - Volume 2 Gary Cornell, Cay S. Horstmann/ Pearson
7. Beginning Java 2 by Ivor Horton; Wrox Publication

Session 1 & 2: (4T+ 2L+ 2SL)
Theory

- Introduction to JVM Architecture
- Java Basics: Overview of Java, Features of Java, Scope of variables
- Object Oriented Concepts
- JDK and its usage (Java Compiler, Java Runtime, Java Debugger, Java doc)
- Working with Data Types: Structure of a Java Class, Importing Packages
- Wrapper classes (Boolean, Double and Integer)
- Operators (Unary, Binary, Arithmetic, Assignment, Compound, Relational, Logical, Equality) and Control Statements (if, if-else, for, while, switch, do-while, break and continue)

Self-Learning:

- Demonstrate when to use for, while and do-while with suitable examples
- Explore enhanced for-loop (for-each) and its limitations, variable scope in nested loops

Suggested Teaching Guidelines for

Advanced Programming for AI (Java Programming)

PGCP-AI February 2026

Session 3 & 4: (6T + 6L + 4SL)
Theory

- Packages
- Arrays
- Understanding of String Class, String Builder Class
- Methods and Encapsulation: Methods, Access Modifiers, Method Overloading, Passing Data, Creating Constructors, Immutable Classes
- Class Inheritance, Abstract Classes, Interface and Implementation classes.
- Understanding Polymorphism: Object vs Reference, Object Casting, Virtual Methods, Method Overriding

Self-Learning:

- Create pattern programs (pyramid, diamond, Pascal's triangle) using nested loops and 2D arrays
- Implement basic sorting algorithms (bubble sort, selection sort) on arrays
- Understand how objects are stored in memory (stack vs heap visualization)
- Understand difference between Object and Reference, Up-casting and down-casting
- Understand how OOP helps in creating modular applications
- Explore SOLID principles of program designing

Session 5: (4T +4L +2SL)
Theory

- Exception-Handling: Basics, Role of Exceptions, Types
- Using try and catch, Multiple Catch, Nested try (throw, throws, finally)
- Built-in Exceptions, Runtime Exceptions Checked Exceptions, Errors
- Creating own Exception Subclasses

Self-Learning:

- Understand the difference between throw and throws keyword
- Try working with multiple catch blocks
- Understand the benefit of using try with resources block
- Implement a custom exception hierarchy for an e-commerce application (PaymentException, InventoryException, ShippingException)
- Checked vs unchecked exceptions - when to use which
- Using Try-with-resources with multiple resources

Suggested Teaching Guidelines for

Advanced Programming for AI (Java Programming)

PGCP-AI February 2026

Session 6: (2T +2L +2SL)

Theory

- Enumerations, Auto boxing
- Java API: java.util, java.lang, java.math
- Generics and Collections - Overview

Self-Learning:

- Understand real-world use of Enums (status, roles) and autoboxing in enterprise applications
- Identify frequently used classes from java.lang, java.util, and java.math in backend systems
- Learn why Generics and Collections are essential for type-safe, scalable data handling
- Explore Collection methods introduced in Java 9

Session 7 & 8 (4T + 4L +4SL)

Theory

- Functional Programming Overview
- Functional Interfaces
- Explore java.util.function package : Predicate, Map, Consumer, Supplier
- Lambda Expressions
- Impact of Functional programming upon Collection Framework

Lab Assignment:

Create an appropriate data structures to store your employee object and use the java.util.package properties.

Self-Learning:

- Understand why functional programming improves readability and maintainability
- Use functional interfaces (Predicate, Function, Consumer, Supplier) in data processing
- Apply lambda expressions to simplify collection operations
- Observe how functional programming enhances Collection Framework performance

Session 9 & 10 (6T + 6L + 4SL)

Theory

- Introduction to Streams
- Streams vs. Collections
- java.util.stream.Stream API
- Types of Primitive Streams : IntStream,LongStream,DoubleStream & its API
- Different operations on streams : filter, map, reduce, sort, flatMap, anyMatch, count, boxing.

- Overview of Java Date Time API

Self-Learning:

- Learn why Streams are preferred over loops for data transformation
- Apply key stream operations (filter, map, reduce, flatMap) for real-world data handling
- Use primitive streams for performance optimization
- Stream vs Parallel Stream - Understand which operations are affected by Parallel stream
- Understand modern Java Date-Time API usage in production systems

Session 11 & 12: (4T + 4L +2SL)

Theory

- Java Concurrency: Using threads in Java, Life cycle of thread
- Advantages and issues
- Thread class, thread groups
- The Runnable interface
- Synchronization

Self-Learning:

- Understand the need for multithreading in responsive and scalable applications
- Learn thread lifecycle, Thread vs Runnable, and common concurrency issues
- Apply synchronization to protect shared resources
- Explore Concurrent collections in Java

Session 13: (2T +2L)

Theory: Reflection in Java

- Why Java Reflection
- Basic Reflection API for finding out details of the class name, super classes & interfaces.

Session 14: (2 T + 2SL)

Theory: Introduction to Node

- Node.js
- Browser JS vs. Node.js
- Node.js REPL

Understand how reflection enables frameworks like Spring and Hibernate

Learn to inspect class metadata, methods, and interfaces at runtime

Identify performance and security considerations of reflection

Self-Learning:

- Understand Node.js as a server-side JavaScript runtime
- Compare Browser JavaScript vs Node.js execution model
- Use Node.js REPL for quick testing and learning

Session 15: (2 T + 2SL)
Theory: Introduction to Spring Frame Work

- What is Spring Framework
- Overview of Spring Architecture
- Spring MVC architecture
- Spring Modules Overview

Self-Learning:

- Understand why Spring is widely used in enterprise Java applications
- Learn the basics of Spring MVC architecture and request flow
- Identify key Spring modules and their roles in backend development